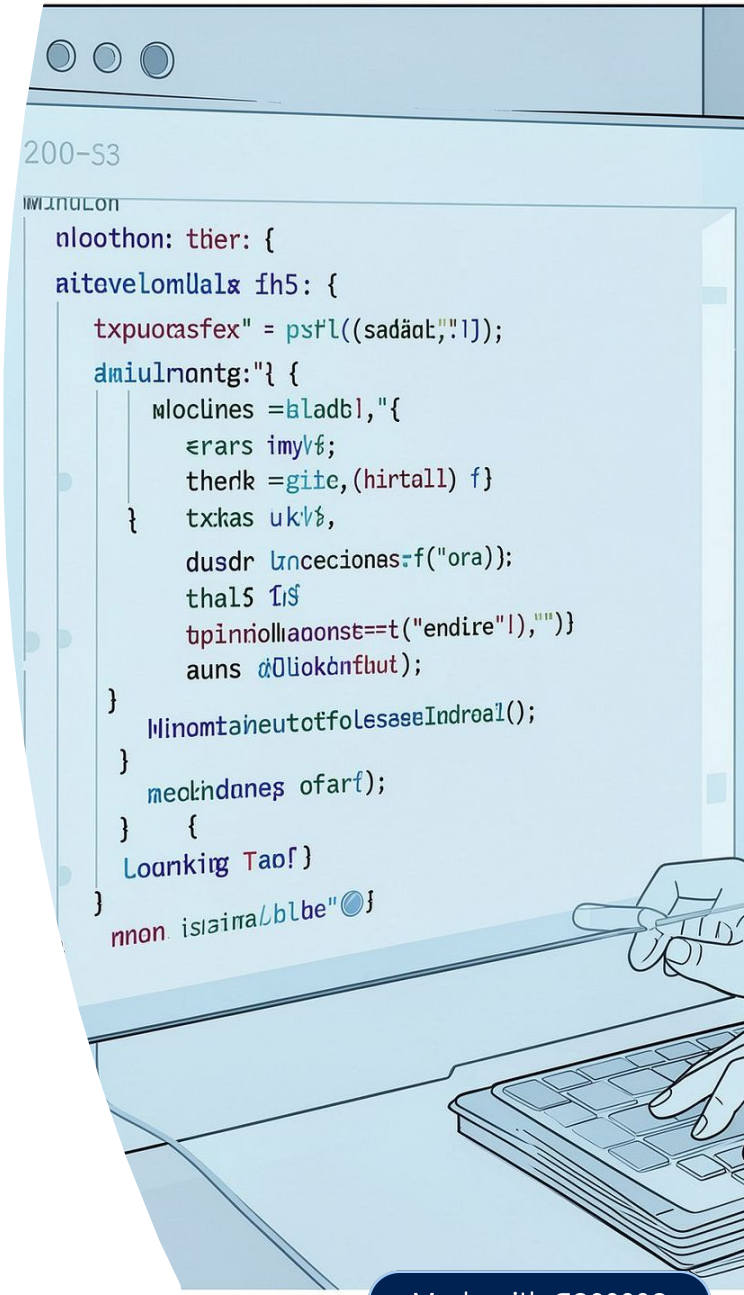
IFSP SÃO CARLOS · TÓPICOS EM ALGORITMOS · PROF. DR. RODRIGO ELIAS BIANCHI · 2026

Longest Substring Without Repeating Characters

Desafio Técnico de Processos Seletivos em Big Techs — LeetCode #3. Otimização assintótica de **O(N³) para O(N) Estrito**, benchmark experimental de CPU e análise rigorosa da Janela Deslizante.

Equipe: Rafael Feltrim (Desenvolvimento & Apresentação) · Ian Fernandes (Pesquisa & Algoritmos) · Gustavo Contiero (Análise de Complexidade & Implementação C)



Made with GAMMA



01 · RELEVÂNCIA DE MERCADO

Por que as Big Techs cobram este problema?

Principal divisor de águas entre quem conhece sintaxe e quem domina **otimização de estruturas de dados e complexidade assintótica**.

> "Em entrevistas técnicas globais, o código representa apenas 30% da avaliação. Os outros 70% medem clareza de decomposição lógica, comunicação e justificativa matemática de Big-O."

Empresas com Aplicação Recorrente

- Amazon (Top 10 para SDE)
- Meta / Facebook (triagem 45 min)
- Google (Níveis L3 e L4)
- Microsoft, Apple e Bloomberg

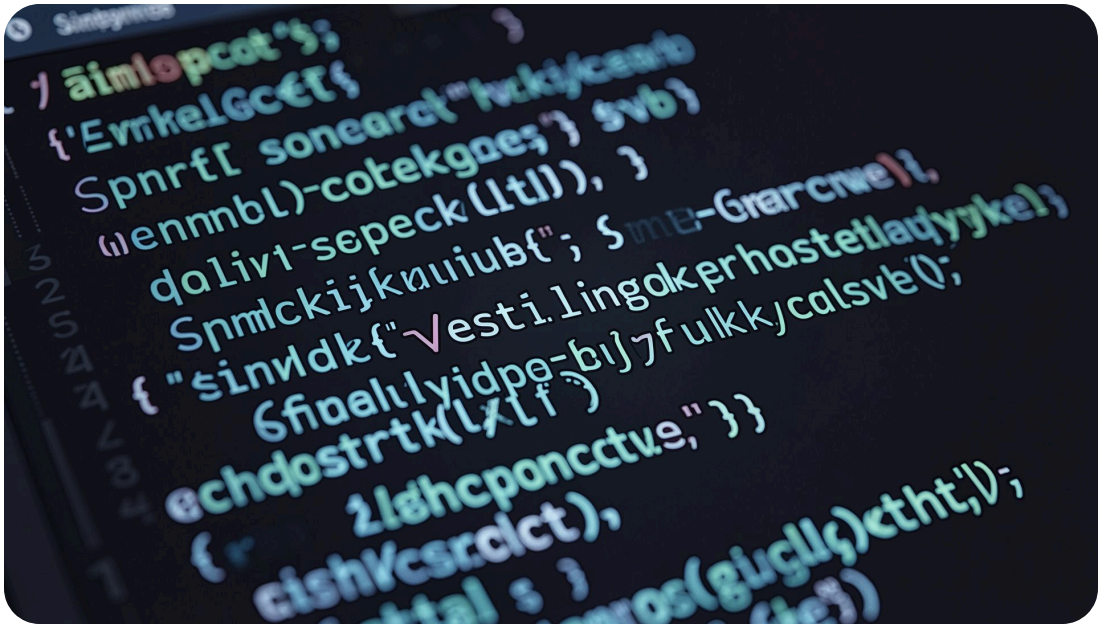
Competências Verificadas

- Eliminar gargalo cúbico $O(N^3)$
- Hash Tables e Arrays $O(1)$
- Casos de borda e alocação estática

Made with GAMMA

02 · DEFINIÇÃO FORMAL

Compreendendo o Problema Computacional



Enunciado Oficial

Dada uma string `s`, encontre o comprimento da **maior substring contígua** sem caracteres repetidos.

Distinção Conceitual

Substring: Sequência estritamente contínua na memória física (ex: `"abc"` em `"abcde"`).

Subsequência: Mantém a ordem relativa, sem continuidade obrigatória (ex: `"ace"` em `"abcde"`).

Casos de Teste

- `s = "abcabcbb"` → **3** (`"abc"`)
- `s = "bbbbb"` → **1** (`"b"`)
- `s = "pwwkew"` → **3** (`"wke"`)

Made with GAMMA

03 · SOLUÇÕES COMPARADAS

Evolução Algorítmica em 3 Níveis de Eficiência

1	2	3
Força Bruta — O(N^3) Gera todas as fatias em O(N^2) e converte cada uma em O(N). **Rejeitado (Time Limit Exceeded).**	**Sliding Window com Set — O(2N)** Dois ponteiros dinâmicos expandindo e encolhendo. Cada caractere entra e sai uma vez. **Contratado (Hire).**	**Sliding Window com Map — O(N) Estrito** Salto direto do ponteiro `left` em O(1) sem loops internos. **Padrão Ouro (Strong Hire).**

Made with GAMMA

04 · RASTREIO DE EXECUÇÃO

Simulação da Memória — s = "abcabcbb"

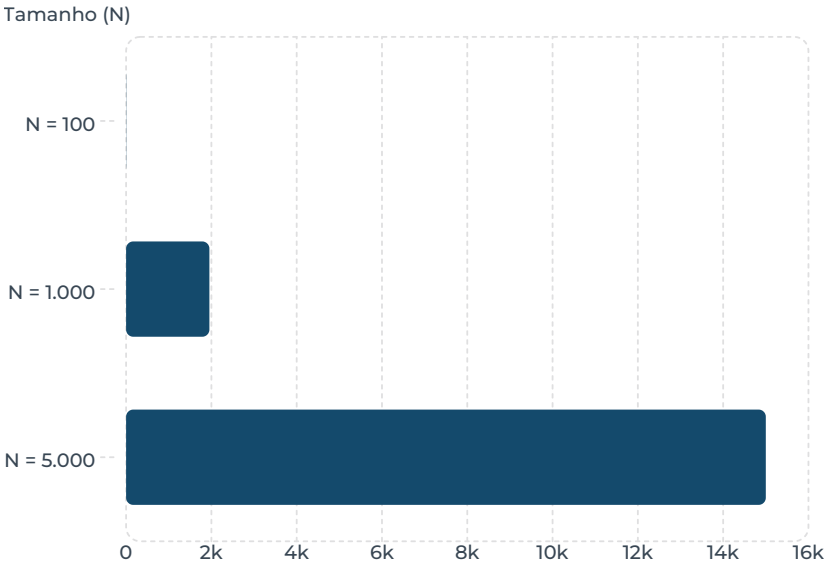
Passo	Char	R	L	Janela	Evento	char_map	max_len
1	'a'	0	0	"a"	1ª ocorrência	{'a':0}	1
2	'b'	1	0	"ab"	1ª ocorrência	{'a':0,'b':1}	2
3	'c'	2	0	"abc"	1ª ocorrência	{'a':0,'b':1,'c':2}	**3 (Pico)**
4	'a'	3	0→1	"bca"	'a' repetido: L salta para 1	{'a':3,'b':1,'c':2}	3
5	'b'	4	1→2	"cab"	'b' repetido: L salta para 2	{'a':3,'b':4,'c':2}	3
6	'c'	5	2→3	"abc"	'c' repetido: L salta para 3	{'a':3,'b':4,'c':5}	3
7	'b'	6	3→5	"cb"	'b' repetido no idx 4: L salta para 5	{'a':3,'b':6,'c':5}	3
8	'b'	7	5→7	"b"	'b' repetido no idx 6: L salta para 7	{'a':3,'b':7,'c':5}	3

Resultado Final: Comprimento Máximo = **3**

Made with GAMMA

05 · BENCHMARK EXPERIMENTAL

Economia de Tempo de CPU — Dados Reais



Métrica de Alto Impacto (N = 50.000)

> 99,99999% de redução

No tempo de processamento total.

416 bilhões × menos ciclos

De instruções de CPU executados.

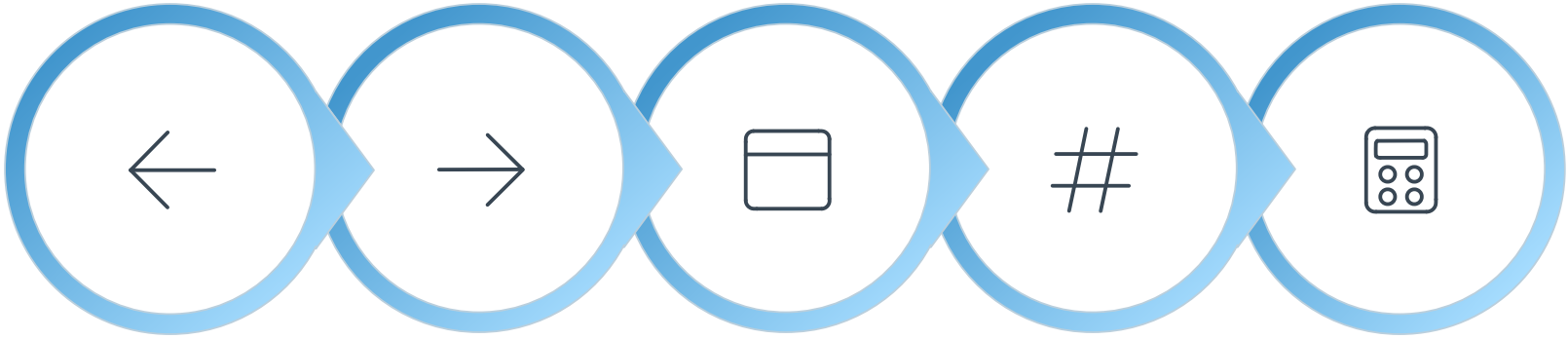
De 4 horas para 11 ms

De computação real em N = 50.000.

Made with GAMMA

06 · ARQUITETURA VISUAL

Mecânica dos Dois Ponteiros — Passo 4



L=1 **R=3** **Janela** **Mapa** **Tamanho=3**

Estado no Passo 4: `left = 1`, `right = 3`, janela ativa `"bca"` (tamanho = 3). O ponteiro esquerdo saltou de 0 para 1 após detectar a repetição de `'a'` via consulta O(1) no `char_map`.

Fórmula da Janela

$$\text{Tamanho} = R - L + 1 = 3 - 1 + 1 = 3$$

char_map Atualizado

`'a'→3` · `'b'→1` · `'c'→2`

Made with GAMMA

07 · IMPLEMENTAÇÃO EM C

Aritmética de Ponteiros e Gestão de Memória na Stack

```
int ultima_pos[256];
for (int k = 0; k < 256; k++)
    ultima_pos[k] = -1;

char *esquerda = s;
int melhor_tam = 0;

for (char *direita = s; *direita != '\0'; direita++) {
    unsigned char c = (unsigned char)*direita;
    int idx_dir = (int)(direita - s);

    if (ultima_pos[c] != -1 &&
        ultima_pos[c] >= (int)(esquerda - s))
        esquerda = s + ultima_pos[c] + 1;

    ultima_pos[c] = idx_dir;
    int tam = (int)(direita - esquerda) + 1;
    if (tam > melhor_tam) melhor_tam = tam;
}
```

Vantagens de Engenharia

Aritmética de Ponteiros Pura

Índices por deslocamento direto `(direita - s)` sem overhead.

Espaço O(1) na Stack

`int ultima_pos[256]` = 1.024 bytes. Zero `malloc()`, zero *memory leak*.

Eficiência de Cache L1

Memória sequencial contígua sem dispersão na Heap.

Made with GAMMA

08 · ANÁLISE DE COMPLEXIDADE

Fundamentação Teórica Assintótica

Abordagem	Tempo	Espaço	Classificação
Força Bruta	$O(N^3)$	$O(\min(N, \Sigma))$	Rejeitado (Red Flag)
Sliding Window com Set	$O(2N) = O(N)$	$O(\min(N, \Sigma))$	Contratado (Hire)
Sliding Window com Map	$O(N)$ Estrito	$O(\min(N, \Sigma))$	Forte Candidato (Strong Hire)

Justificativa de Tempo: O(N)

O cursor direito percorre a sequência de 0 a $N - 1$ exatamente uma vez. Consultas e atualizações de índice ocorrem em tempo constante **O(1)**.

Justificativa de Espaço: O(min(N, Σ))

A tabela auxiliar armazena no máximo o menor valor entre caracteres únicos da string N e o tamanho do alfabeto Σ (ASCII = 256).

Made with GAMMA

09 · CASOS DE BORDA & CONCLUSÃO

Cenários Extremos Validados

String Vazia `""`

Retorna 0 imediatamente.

Caracteres Idênticos `"bbbbbb"`

Janela se mantém em tamanho 1.

Todos Distintos `"abcdef"`

Janela expande linearmente até N.

Caracteres Especiais `"a b c!"`

Tratados nativamente pela tabela ASCII de 256 bytes.

Problema

Longest Substring Without Repeating Characters — LeetCode #3

Solução Ótima

Janela Deslizante com Hash Map e Aritmética de Ponteiros em C

Ganhos

De $O(N^3)$ para $O(N)$ — economia de > 99,99999% de CPU

Repositório GitHub e Vault Obsidian totalmente sincronizados. Agradecemos a atenção do Prof. Bianchi e dos colegas. Abrimos para perguntas da banca.

Made with GAMMA